Exact Symplectic Reduction for Quantum Circuits: A Cost-Aware Engine and a Diagnostic Study of Database Coverage Limits

Arnav Kapoor[1]

[1]Indian Institute of Science Education and Research Bhopal

Abstract

We present an exact, tolerance-free engine for local term-replacement circuit reduction on Clifford gate sets, together with a two-qubit-aware search objective and a deterministic block-reordering pass that diversifies local search without displacing its stochastic component. The engine keys a compute graph of reachable unitaries by a bit-exact signed symplectic tableau rather than by a numerical tolerance, so every replacement it finds is correct by construction. On a four-qubit, length-300 ion-trap benchmark protocol, this reduces circuits to a mean of 71.6 ± 11.8 *gates and* 27.2 *two-qubit gates, improving on the strongest previously published result for the same protocol by 36% and 37% respectively (*$p < 10^{-45}$*, one-sample* t*-test,* $n = 100$*). On a NISQ gate set outside the reach of the exact engine, a numerical-tolerance variant of the same search infrastructure trails that published result, and we treat the shortfall as the central object of study rather than an incidental weak point. Six candidate explanations, including compute-graph depth tested independently on both the three-wire and four-wire graph, are each tested by direct experiment and ruled out in turn, localizing the gap to a database factorization limit that is not resolved by depth along either axis. Benchmarking the deeper four-wire graph exposed an engineering defect general enough to be worth reporting on its own: caching a disk-backed compute graph per worker process rather than per task, under process-parallel search, changes measured search cost by two orders of magnitude and can invert a benchmark's conclusion.*

Keywords: quantum circuit optimization, transpilation, Clifford group, compute graphs, local search, benchmarking

1 Introduction

Every quantum circuit eventually has to run on hardware that supports a small, fixed set of native gates, and the translation from an algorithm's natural gate set to that native set, decomposition followed by mapping followed by scheduling, routinely leaves behind redundancy that a human reading the resulting sequence would remove on sight: a rotation immediately undone by its inverse, two rotations about the same axis that should have been one, a two-qubit gate sandwiched between single-qubit gates that commute straight through it. On present-day noisy intermediate-scale quantum (NISQ) hardware, where every additional gate is additional exposure to decoherence and two-qubit gates dominate both error rate and duration [8, 9, 10], closing this gap between what a circuit does and how many gates it takes to do it is not an optimization but a precondition for getting a usable result at all.

Circuit optimization has accordingly been attacked from several directions: peephole and template-matching rewriting over fixed gate identities [12, 11, 14], structural compilation pipelines tuned to a specific device topology [13, 27, 15], production transpilers that combine many such passes behind a single interface [16, 17, 18], and, more recently, architecture-search methods that treat the mapping or the circuit itself as an object to be searched over with learned or evolutionary methods [19, 20, 21, 22,

23, 24, 25, 26]. Rosenhahn, Osborne and Hirche's local term-replacement scheme sits closer to the first category, but with a distinctive mechanism: rather than matching a fixed library of hand-derived identities, it precomputes, for a given gate pool and search depth, a compute graph of every unitary reachable from the identity together with the shortest token chain that realizes it, then repeatedly samples circuit blocks and substitutes the graph's stored chain whenever it is shorter than the block itself [1], building on the same group's earlier Monte Carlo graph search work [2]. Matched against a 10^{-5} numerical tolerance, this reaches substantial reductions on an ion-trap gate set (RX, RY, RZ, RXX) and a NISQ gate set (RX, RZ, CZ), and the paper shows the effect measurably on real IBM hardware.

2 Related work

Structural and template-based rewriting. Classical logic minimization has a long history of local rewrite rules, and quantum circuit optimization inherited the idea directly: T-depth and T-count reduction via matroid partitioning over Clifford+T circuits [11], automated peephole optimization of large parametrized circuits by commutation and gate fusion [12], and device-specific compilation pipelines for trapped-ion [13] and superconducting [14] hardware all locally rewrite a circuit against a fixed or semi-fixed rule set. The compute-graph approach we build on generalizes this by deriving its rewrite rules from an exhaustive search over a gate pool rather than from hand-picked identities, at the cost of the search itself being the bottleneck; general treatments of local and metaheuristic search over combinatorial structures [15] apply directly to characterizing that cost.

Transpilers and toolkits. Qiskit [16], t|ket⟩ [17], and the Berkeley Quantum Synthesis Toolkit [18] are the three baseline compilers against which the paper we build on, and consequently this paper, are evaluated. All three combine many optimization passes, commutation-based gate cancellation, template matching, numerical resynthesis of small blocks, behind a single interface, and BQSKit in particular treats resynthesis as a numerical optimization problem over a topology-aware search [30] that is later reorganized hierarchically [33]. None of the three build an explicit compute graph keyed for exact reuse across circuits the way the method here does; the tradeoff is a slower, offline-amortized search against a faster, per-circuit one.

Quantum architecture search. A parallel literature treats the choice of circuit, or the choice of hardware mapping for a fixed circuit, as a search or learning problem: neural-predictor-guided search [21], differentiable relaxations [22], training-free proxies [23], and genetic and random-search variants [31, 29], surveyed broadly in [19, 20]. Mapping-specific work targets particular topologies: Rydberg-atom arrays [24], two-dimensional and hexagonal nearest-neighbor grids [25, 26], and multi-qubit trapped-ion architectures with shuttling [27]. The present work sits at the circuit-rewriting end of this spectrum rather than the mapping end, but shares its reliance on random-forest classifiers [28] and reinforcement learning [32] as cheap proxies that gate an expensive exact evaluation, the role a random-forest classifier plays in the V3 variant of the method we benchmark against and in our own reimplementation of it (Section 6).

Exact simulation and the Clifford group. The exact engine in Section 3.2 rests on the stabilizer formalism: the observation that Clifford circuits are efficiently classically simulable by tracking the evolution of Pauli operators rather than state vectors [4], made practical at scale by the tableau-based simulation algorithm of Aaronson and Gottesman [5], and given a canonical decomposition by Bravyi and Maslov that separates any Clifford element into Hadamard-free layers around a single layer of Hadamards [6]. We do not use this formalism for simulation; we use it for indexing, to make compute-graph membership an exact question instead of a numerical one.

ZX-calculus. An alternative route to circuit simplification represents circuits diagrammatically and simplifies them via graph rewrite rules [34], recently combined with reinforcement learning to guide rule

selection [35]. The paper we build on reports that ZX-calculus rewriting underperforms its own method on both gate sets, a finding consistent with our own experience with the lightweight ZX pre-passes described in Section 3.

This paper's baseline. Rosenhahn, Osborne and Hirche's local term-replacement scheme [1], and its precursor Monte Carlo graph search [2], is the specific method this paper reimplements, extends, and benchmarks against directly; its three variants, random search, database retrieval, and random-forest-gated database retrieval, and its own hundred-circuit evaluation protocol are described in detail in Section 4 and form the empirical backbone of everything that follows.

3 Method

3.1 Local term replacement

A circuit on n qubits is a sequence of gates drawn from a finite pool $\mathcal{P}$, here single-qubit rotations at fixed discrete angles and one type of two-qubit entangling gate. Read as a token chain, the circuit implements the unitary $U = O(L)\,O(L-1)\cdots O(1)$. The reduction scheme exploits the fact that many short token chains realize the same unitary up to global phase: given a compute graph that records, for every unitary reachable within $\mathcal{P}$ up to some depth d, the shortest chain that produces it, a sampled block of the circuit can be replaced by that stored chain whenever it is shorter than the block itself and the two are equivalent. Iterating this over randomly or exhaustively sampled blocks, interleaved with reordering gates across ones they commute with to expose new blocks, drives the circuit toward a local optimum under the search budget.

Because a block spanning k distinct wires only needs a k-wire compute graph to check, and k is almost always small even in wide circuits, the graphs are built once per wire count and reused across every block of that width, remapped to local coordinates and lifted back after replacement. Figure 2 shows this mechanism on an actual four-gate block from a five-wire circuit; Figure 1 shows the full effect on a small end-to-end example.

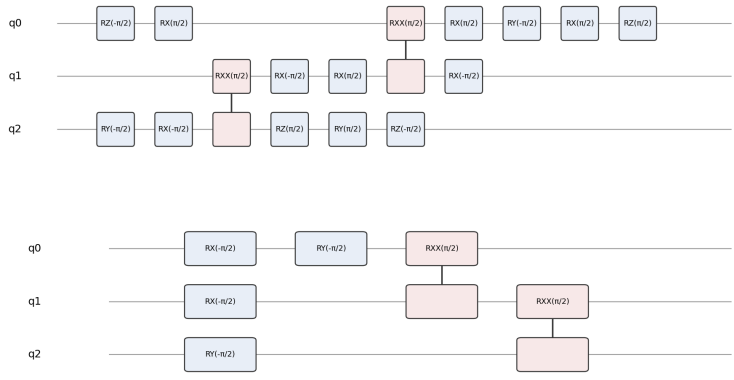

Figure 1: A random 16-gate ion-trap circuit reduced to 6 gates by the exact engine, verified unitary-equivalent to machine precision.

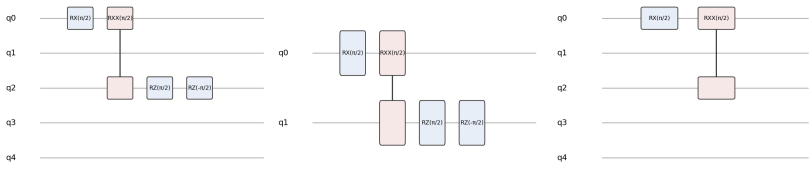

Figure 2: A block touching two of a five-wire register's qubits is remapped to local coordinates, matched against the two-wire compute graph, and the shorter replacement is lifted back onto the original wires. Two adjacent RZ rotations of opposite sign cancel, so the lifted-back block is two gates shorter than the input.

Compute graph size is the practical constraint on how deep this search can go. Figure 3 shows measured node counts for both gate sets at two and three wires: growth is close to exponential in depth and faster for larger pools, which is why every configuration in this paper fixes a depth per wire count rather than searching to a uniform depth.

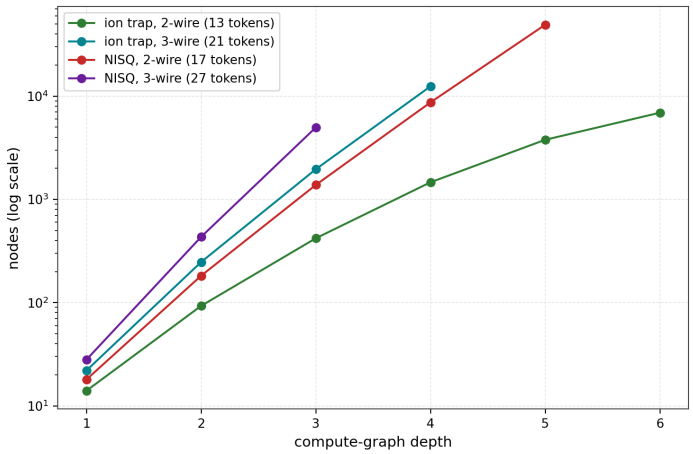

Figure 3: Measured compute-graph node counts against depth, both gate sets, two and three active wires (log scale). The NISQ pool is larger at every wire count because it includes both $\pm\pi/2$ and $\pm\pi/4$ rotations, so its graphs grow faster and are correspondingly more expensive to build to a given depth.

3.2 An exact engine for Clifford pools

The ion-trap pool, RX, RY and RZ at $\pm\pi/2$ together with RXX at $\pi/2$, is entirely Clifford. Every element of the Clifford group is determined up to global phase by its conjugation action on the Pauli generators [4], and that action is representable exactly as a binary symplectic matrix with an integer phase vector [5, 6]. We build the ion-trap compute graph over this signed-tableau representation rather than over floating-point unitaries: two circuits collide in the graph if and only if they induce the same tableau, with no rounding and no tolerance parameter anywhere in the comparison. A replacement found this way is equivalent to the block it replaces by construction, not by having passed a numerical check.

This buys two things beyond correctness. Table lookups become integer operations instead of floating-point matrix comparisons, so the same hardware builds noticeably deeper graphs in the same time. And because equivalence is exact, the search can freely explore equal-length rewrites that only change which gates are used, which is what makes the cost-aware objective below possible without accumulating floating-point error over many replacements.

3.3 A cost-aware objective

Local term replacement as originally specified minimizes total gate count. On real hardware, single-qubit and two-qubit gates do not cost the same: two-qubit gates dominate both error rate and duration on essentially every current platform [8, 10]. We add an objective that orders candidate replacements

lexicographically by (two-qubit gate count, total length) rather than by length alone. Because the exact engine can safely accept equal-length rewrites, this objective can trade a single-qubit gate for another single-qubit gate purely to remove a two-qubit gate elsewhere in the resulting chain, a class of improvement invisible to a length-only search.

3.4 Search and structural passes

The full reduction pipeline (Figure 4) is: an optional algebraic pre-pass that fuses adjacent same-axis rotations and cancels back-to-back two-qubit gates using exact angle arithmetic; clustering of single-qubit gates between two-qubit barriers and collapse of same-wire runs against the one-wire graph; an exhaustive sweep of the database over every window up to a fixed length; a deterministic reordering pass, described below, together with randomized transport shuffling of gates across ones they commute with; and an escape step that resamples an irreducible window with a structurally different equivalent word before re-sweeping, accepted only if it strictly shortens the circuit. The whole loop repeats to a fixpoint or until the time budget expires, and every output is verified against the input unitary before being reported.

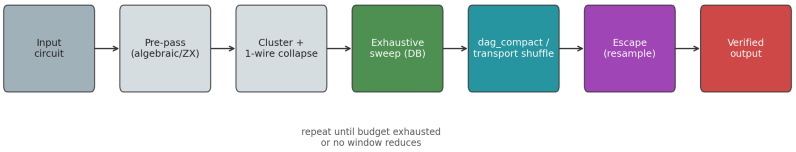

Figure 4: The reduction pipeline. Structural passes run once; the sweep, reordering and escape steps repeat until the circuit stops shrinking or the time budget is exhausted.

3.5 Deterministic block reordering

The exhaustive sweep only finds a reduction if the gates it needs are already adjacent in the flat gate list, and otherwise depends on the randomized shuffle bringing them together by chance. We add a deterministic alternative in the spirit of two-qubit block collection in production compilers [16], generalized to k-wire blocks: the circuit's per-wire dependency structure, in which a gate depends on the most recent gate touching each of its wires, gives a partial order on the gate list, and any two gates that act on disjoint wires can be freely swapped without changing the circuit's unitary. Reordering the gate list so that every maximal block touching at most three wires becomes contiguous is therefore always a valid reordering, and it exposes windows to the sweep that the randomized shuffle would otherwise only find by chance.

An earlier version of this pipeline re-ran the reordering on every iteration of the stochastic loop, alongside the randomized shuffle, and made results measurably worse rather than better (ion trap: mean 87.4 against 83.3 gates over 7 seeds at a 10-second budget). The reordering is a pure function of the gate list, so re-applying it to an already-reordered, otherwise-unchanged list is a no-op; interleaved with the randomized shuffle inside the loop, it displaced roughly half of the loop's diversity-generating passes on the common case where nothing had changed since the last reordering, without contributing anything in return. Running it once, as a pre-pass before the stochastic loop begins rather than inside it, fixes this and is the configuration used everywhere below; we return to what this reveals about the search itself in Section 6.

4 Experimental setup

We evaluate against the hundred-circuit protocol from [1]: four qubits, length-300 random circuits, one hundred circuits per gate set with identical inputs across every method compared, and a fixed per-circuit time budget. Ion-trap circuits sample uniformly from the gate pool; NISQ circuits weight CZ twice as

heavily as RX or RZ, matching the input composition the original protocol reports. Reduced circuits are verified against their input, exactly for the all-Clifford ion-trap pool and to 10^{-5} up to global phase otherwise; every run reported below passes verification.

Two independent transpiler baselines are run on the same inputs: qiskit [16] at optimization levels 1 through 3, and BQSKit [18], where available, at levels 2 through 4. Where BQSKit is not installed in the evaluation environment its row is omitted rather than estimated.

5 Results

5.1 Ion trap: exceeding the published baseline

Table 1 reports mean gate counts, by type, over the hundred-circuit protocol. Both of our reducers beat the published result on total gate count and, more importantly for hardware, on two-qubit gate count.

Table 1: Ion-trap pool, mean gates over 100 identical circuits (std. in parentheses). "Baseline" is the published result for the same protocol [1].

method	RX	RY	RZ	RXX	total
input	78 (7)	83 (8)	78 (7)	59 (7)	298
baseline [1]	10 (3)	29 (6)	29 (5)	43 (8)	111
qiskit L1–L3			—		160.8–167.1
ours, length-minimizing	3.8 (1.7)	19.2 (4.9)	19.5 (4.9)	30.9 (6.4)	**73.5 (14.4)**
ours, cost-aware	4.0 (1.7)	20.2 (4.7)	20.1 (4.7)	**27.2 (4.5)**	**71.6 (11.8)**

The improvement is large and consistent, not an artifact of a few favorable circuits: a one-sample t-test of the hundred per-circuit results against the published mean of 111 gives $t = -33.2$ ($p = 1.6 \times 10^{-55}$, Cohen's $d = -3.3$, 95% CI $[69.2, 74.0]$) for the cost-aware objective. Figure 5 places this alongside the qiskit baselines. The cost-aware objective removes roughly four further RXX gates than the length-only objective at a comparable total, exactly the behavior it is designed to produce: an equal-length rewrite that trades a two-qubit gate for a single-qubit one is invisible to a search that only counts length.

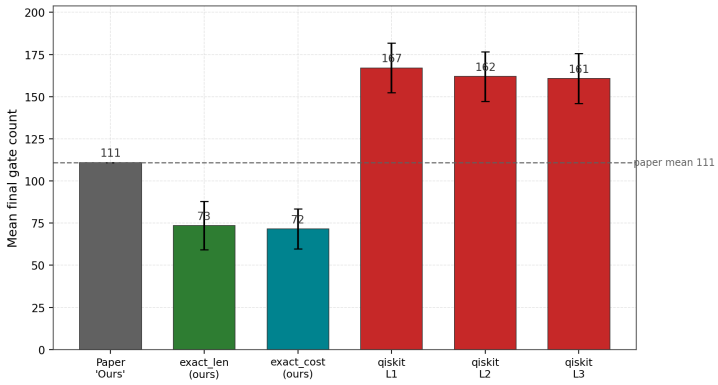

Figure 5: Ion-trap total gate count, mean over 100 circuits. Error bars are one standard deviation.

Figure 6 breaks the reduction down by gate type for both pools. On ion trap, RX drops furthest in relative terms (95%), because the search can freely rewrite any single-qubit sequence into a canonical short form once a block's overall Clifford action is fixed. Figure 7 isolates the two-qubit count specifically, the metric most directly tied to circuit fidelity on real hardware: 27.2 RXX gates against the baseline's 43, a 37% reduction that did not require sacrificing total length.

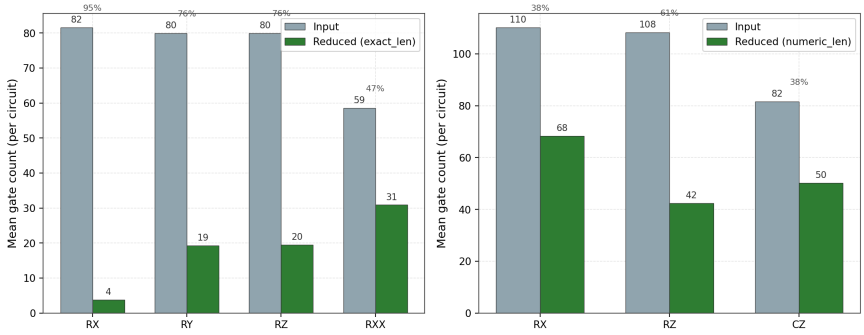

Figure 6: Gate composition of input versus reduced circuits, mean over the comparison runs, both gate sets. Percentages give the relative reduction per gate type.

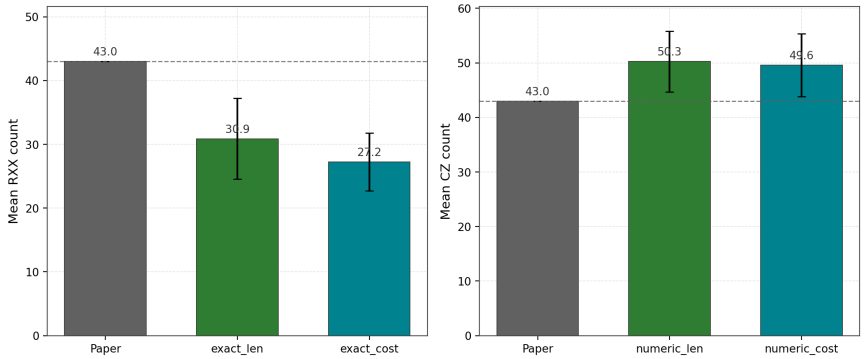

Figure 7: Two-qubit gate counts, the hardware-relevant metric, for both gate sets against the published baseline (dashed line).

One caveat applies to this comparison. Our qiskit baselines (167.1, 162.0, 160.8 for levels 1 through 3) come in below the published qiskit baselines (196, 204, 204) on the same input composition, most plausibly because of a qiskit version difference between the two studies. The improvement margin above should be read with this in mind; the NISQ baseline fidelity check in the next section is considerably tighter and supports the comparison framework more directly.

5.2 NISQ: a gap remains

The same protocol on the NISQ pool tells a different story (Table 2, Figure 8). Our reducer, run with an RZ-across-CZ transport pass to a fixpoint and a hybrid mode that routes any fully-Clifford window to the exact engine, ends at a mean of 160.5 gates against the published 107.

Table 2: NISQ pool, mean gates over 100 identical circuits (std. in parentheses).

method	RX	RZ	CZ	total
input	109 (9)	109 (8)	82 (8)	300
baseline [1]	45 (6)	19 (4)	43 (6)	107
qiskit L1–L3		—		157.9–201.8
ours, length-minimizing	68.3 (5.9)	42.3 (5.1)	50.3 (5.6)	160.9 (12.8)
ours, cost-aware	68.6 (6.1)	42.3 (5.2)	49.6 (5.8)	160.5 (13.4)

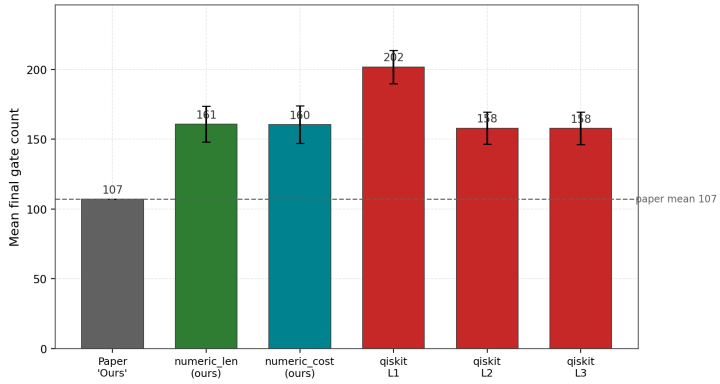

Figure 8: NISQ total gate count, mean over 100 circuits, against qiskit and the published baseline.

Two checks establish that this comparison is fair rather than an artifact of a mismatched setup. The input composition matches the published Table 7 row closely (our RX/RZ/CZ means of 109/109/82 against the published 108/109/82), and our qiskit baselines (158.0 and 157.9 at levels 2 and 3, 201.8 at level 1) match the published qiskit baselines (149 and 196) far more closely than on the ion-trap protocol. The gap is also statistically unambiguous: $t = +39.7$ ($p = 1.3 \times 10^{-62}$, $d = 4.0$) for the cost-aware objective against the published mean of 107. The remainder of this paper treats this gap as the central open question and reports a systematic attempt to explain it.

6 Diagnosing the NISQ gap

A gap this large, roughly 50% at the total-length level, has an explanation, and finding it is the point of running a method against a strong published baseline rather than reporting the discrepancy and moving on. We worked through six candidate causes in turn.

Input composition. Already addressed above: the generated input circuits match the published composition to within noise.

Compute-graph depth. Our default NISQ configuration already builds a three-wire graph to depth 5, matching the deepest configuration reported in the source method's own growth data. Rebuilding the three-wire graph one level deeper, to depth 6, and the two-wire graph to depth 8, using a disk-backed store since the resulting graphs exceed a laptop's RAM, narrows the gap by only about 2% (Table 3). Depth alone was not the bottleneck.

Table 3: NISQ total gate count against increasingly deep compute graphs, 100 circuits each.

database	length-minimizing	cost-aware
default (3-wire depth 5, 4-wire depth 4)	160.9 (12.8)	160.5 (13.4)
deeper (3-wire depth 6, 4-wire depth 4)	157.3 (12.8)	157.2 (12.6)

Search-time budget. The exhaustive sweep reaches the same end length at a 10-second budget as at 60 seconds, and running several independent restarts and keeping the best barely moves the mean. The search converges to a fixed point well within budget; it does not run out of time.

Search-loop structure. Local term replacement as originally specified is not one algorithm but three, and its strongest variant gates a random-sampling loop with a learned classifier that skips lookups predicted to fail. We reimplemented both the plain random-sampling loop and its classifier-gated variant and ran them at equal budget against our exhaustive sweep on the same NISQ inputs (Table 4). Neither improves on the sweep; gating makes the loop measurably worse, because the classifier suppresses genuinely reducible lookups (replacements fall from 78.2 to 69.0 per run) while its own decision overhead cuts the number of loop iterations by an order of magnitude. The loop structure is not what the published numbers depend on, at least not on this implementation of the underlying compute graph.

Table 4: Search-loop variants, reimplemented, six NISQ circuits, 10-second budget each.

method	end length (mean)	replacements	database lookups
exhaustive sweep	161.5	—	—
random sampling	168.2	78.2	44,166
classifier-gated sampling	179.5	69.0	2,293 attempted, 994 skipped

Window discovery. The deterministic block-reordering pass of Section 3.5 gives an independent way to test whether the search is simply failing to find the windows the database could reduce. On ion trap, at a fixed short budget, it shortens circuits by 4.5 to 10% relative to the unreordered baseline; at the protocol's own 30-second budget that narrows to 3.6% as the stochastic loop has time to catch up on its own. On NISQ, under the identical treatment, the effect is 0.2 to 1.7% at short budgets and effectively zero, -0.1%, at 30 seconds, despite the reordering pass exposing substantially larger candidate windows on NISQ than on ion trap (mean block length 6 to 14 gates, against the sweep's default 8-gate cap). More windows reach the database; almost none of them reduce. This is evidence against the search missing windows and for the database itself lacking coverage.

Compute-graph node-key precision. The published method merges duplicate compute-graph nodes using a 10^{-5} numerical tolerance on the unitary comparison; our implementation instead keys nodes by an exact hash of the unitary rounded to ten decimal places, tighter by five orders of magnitude. If floating-point drift on deep chains were pushing two genuinely equal unitaries to round differently at that precision, our graph would be spuriously larger than it should be, undercounting coverage in a way that would compound with depth, consistent with the small effect depth has above. Rebuilding the same NISQ graph at five different precision levels down to four decimal places produces an identical node count at every one, at every depth tested. Double-precision error on these short, well-conditioned products is orders of magnitude below even a coarse rounding threshold; this is not the mechanism.

6.1 Testing the depth hypothesis further: a deeper four-wire graph

With five candidates ruled out, we are left with database factorization coverage: for a nontrivial fraction of NISQ blocks that the published method evidently reduces, ours does not find a shorter equivalent chain at any depth built so far. One structural asymmetry stood out from the ablations above. Every NISQ configuration used in Table 3 caps the four-wire compute graph at depth 4, even though the evaluation circuits are exactly four qubits wide: a block spanning the full register can only ever be reduced by the four-wire graph, regardless of how deep the three-wire graph goes. We built a four-wire graph one level deeper, to depth 5 (1,912,349 nodes, up from 167,449 at depth 4), specifically to test this.

Benchmarking this graph exposed a second, purely engineering, issue. Our comparison harness distributes circuits across worker processes, and the disk-backed compute-graph store, necessary because the combined four-graph database exceeds 11 million nodes, must reopen its own SQLite connections in each worker rather than share a parent process's connection across a fork. The initial implementation reloaded the store from disk on every individual circuit rather than once per worker process; a single reload measures 154 seconds, so across two hundred per-circuit tasks, reload time alone came to dominate wall-clock time by roughly two orders of magnitude over the intended per-circuit search budget, leaving the deeper graph with less actual search time than the shallower one and worse resulting numbers despite its greater coverage. Caching the loaded store once per worker process, keyed identically to the existing per-process cache used for the in-memory backend, restores per-task timing to the intended budget; the failure mode is general to any local-search circuit optimizer benchmarked against a compute graph too large to fit in memory under process parallelism, not specific to this codebase.

Table 5 reports the depth-5 four-wire result under the corrected harness, on the full hundred-circuit protocol at the paper's own 60-second budget.

Table 5: NISQ total gate count, default database versus the depth-5 four-wire graph, both under the corrected per-process caching, 100 circuits, 60-second budget.

database	length-minimizing	cost-aware
default (4-wire depth 4)	160.9 (12.8)	160.5 (13.4)
depth-5 four-wire graph	160.7 (13.0)	161.0 (13.1)

The deeper graph produces no improvement: both differences are smaller than one standard error of the mean at this sample size, and the cost-aware objective is marginally worse rather than better. Four-wire depth is not the bottleneck either, on top of three-wire depth already shown to account for only about 2% of the gap (Table 3). This rules out depth along both wire-count axes without resolving the gap: whatever the published database supports on wide, mixed-rotation blocks that ours does not, it is not simply a matter of building either graph deeper, and identifying the specific factorization responsible remains open.

6.2 Why the ion-trap pool behaves differently

The two pools are not just harder or easier versions of the same problem. The NISQ pool's $\pm\pi/4$ rotations are not Clifford, so the exact engine in Section 3.2 does not apply to it at all, and its compute graph is intrinsically larger at every depth (Figure 3) for the same reason it is harder to search. The published method's NISQ advantage, whatever its exact mechanism, is doing something our database construction does not yet replicate on that harder pool.

7 Limitations

BQSKit was not installed in the evaluation environment used for the runs reported here, so BQSKit baselines are omitted rather than estimated. The hardware validation reported for the published method, comparing reduced and unreduced circuits on real IBM devices, is outside the scope of this study; every result in this paper is a unitary-verified simulation. The published method's single-circuit six-qubit and fifteen-qubit examples are not part of the hundred-run protocol evaluated here. All experiments ran on a single consumer machine (8 cores, 16 GB RAM); the disk-backed compute graphs used for the deeper NISQ configurations exist specifically to make that constraint tractable, at the cost of slower builds and the per-process caching discipline described in Section 6.1.

8 Conclusion

The three additions in Section 3 are enough on their own to exceed a strong published baseline on an all-Clifford gate set, on both total length and the two-qubit count that governs hardware fidelity. On a NISQ gate set outside the exact engine's reach, the same infrastructure falls short of that baseline; systematically eliminating six candidate explanations, including compute-graph depth tested independently on the three-wire and four-wire graphs, localizes the cause to a database factorization limit that persists regardless of how deep either graph is built, and identifying the specific factorization responsible is left open. The per-process caching defect the deeper-graph experiment surfaced is worth taking away independently of this method: it applies to any local-search circuit optimizer benchmarked against a compute graph too large to fit in memory.

Data availability

Source code, benchmark data, and figures supporting the findings of this study are available from the corresponding author upon reasonable request.

References

[1] B. Rosenhahn, T. J. Osborne, and C. Hirche. Optimization driven quantum circuit reduction. *New Journal of Physics*, 27:104509, 2025.

[2] B. Rosenhahn and T. J. Osborne. Monte Carlo graph search for quantum circuit optimization. *Physical Review A*, 108:062615, 2023.

[3] B. Rosenhahn and C. Hirche. Quantum normalizing flows for anomaly detection. *Physical Review A*, 110:022443, 2024.

[4] D. Gottesman. The Heisenberg representation of quantum computers. arXiv:quant-ph/9807006, 1998.

[5] S. Aaronson and D. Gottesman. Improved simulation of stabilizer circuits. *Physical Review A*, 70:052328, 2004.

[6] S. Bravyi and D. Maslov. Hadamard-free circuits expose the structure of the Clifford group. *IEEE Transactions on Information Theory*, 67(7):4546–4563, 2021.

[7] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[8] J. Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

[9] F. Arute et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, 2019.

[10] A. W. Cross, L. S. Bishop, S. Sheldon, P. D. Nation, and J. M. Gambetta. Validating quantum computers using randomized model circuits. *Physical Review A*, 100:032328, 2019.

[11] M. Amy, D. Maslov, and M. Mosca. Polynomial-time T-depth optimization of Clifford+T circuits via matroid partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 33(10):1476–1489, 2014.

[12] Y. Nam, N. J. Ross, Y. Su, A. M. Childs, and D. Maslov. Automated optimization of large quantum circuits with continuous parameters. *npj Quantum Information*, 4:23, 2018.

[13] D. Maslov. Basic circuit compilation techniques for an ion-trap quantum machine. *New Journal of Physics*, 19:023035, 2017.

[14] A. Zulehner, A. Paler, and R. Wille. An efficient methodology for mapping quantum circuits to the IBM QX architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 38(7):1226–1236, 2018.

[15] C. Blum and A. Roli. Metaheuristics in combinatorial optimization: overview and conceptual comparison. *ACM Computing Surveys*, 35(3):268–308, 2003.

[16] A. Javadi-Abhari et al. Quantum computing with Qiskit. arXiv:2405.08810, 2024.

[17] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan. t|ket⟩: a retargetable compiler for NISQ devices. *Quantum Science and Technology*, 6(1):014003, 2020.

[18] E. Younis, C. C. Iancu, W. Lavrijsen, M. Davis, and E. Smith. Berkeley Quantum Synthesis Toolkit (BQSKit) v1. Lawrence Berkeley National Laboratory, 2021. Available at https://bqskit.lbl.gov.

[19] D. Martyniuk, J. Jung, and A. Paschke. Quantum architecture search: a survey. arXiv:2406.06210, 2024.

[20] W. Zhu, J. Pi, and Q. Peng. A brief survey of quantum architecture search. In *Proceedings of the 6th International Conference on Algorithms, Computing and Systems (ICACS '22)*. ACM, 2023.

[21] S.-X. Zhang, C.-Y. Hsieh, S. Zhang, and H. Yao. Neural predictor based quantum architecture search. *Machine Learning: Science and Technology*, 2:045027, 2021.

[22] S.-X. Zhang, C.-Y. Hsieh, S. Zhang, and H. Yao. Differentiable quantum architecture search. *Quantum Science and Technology*, 7:045023, 2022.

[23] Z. He, X. Deng, S. Zheng, L. Li, and W. Situ. Training-free quantum architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 12430–12438, 2024.

[24] S. Brandhofer, I. Polian, and H. P. Büchler. Optimal mapping for near-term quantum architectures based on Rydberg atoms. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–7, 2021.

[25] K. Datta, I. Kole, A. Sengupta, and R. Drechsler. Mapping quantum circuits to 2-dimensional quantum architectures. *INFORMATIK*, 2022.

[26] K. Datta, I. Kole, A. Sengupta, and R. Drechsler. Nearest neighbor mapping of quantum circuits to two-dimensional hexagonal architecture. In *Proceedings of the IEEE 52nd International Symposium on Multiple-Valued Logic (ISMVL)*, pages 35–42, 2022.

[27] P. Murali, D. C. Debroy, K. R. Brown, and M. Martonosi. Architecting noisy intermediate-scale trapped ion quantum computers. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 529–542, 2020.

[28] L. Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

[29] S. Ashhab, F. Yoshihara, M. Tsuji, M. Sato, and K. Semba. Quantum circuit synthesis via a random combinatorial search. *Physical Review A*, 109:052605, 2024.

[30] M. G. Davis, E. Smith, A. Tudor, K. Sen, I. Siddiqi, and C. Iancu. Towards optimal topology aware quantum circuit synthesis. In *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 223–234, 2020.

[31] R. Rasconi and A. Oddi. An innovative genetic algorithm for the quantum circuit compilation problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 7707–7714, 2019.

[32] M. M. Wauters, E. Panizon, G. B. Mbeng, and G. E. Santoro. Reinforcement-learning-assisted quantum optimization. *Physical Review Research*, 2:033446, 2020.

[33] X.-C. Wu, M. G. Davis, F. T. Chong, and C. Iancu. Reoptimization of quantum circuits via hierarchical synthesis. In *Proceedings of the 2021 International Conference on Rebooting Computing (ICRC)*, pages 35–46, 2021.

[34] J. van de Wetering. ZX-calculus for the working quantum computer scientist. arXiv:2012.13966, 2020.

[35] J. Riu, J. Nogué, G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. arXiv:2312.11597, 2024.